HCM Lite: System Design and Build Document

AI-assisted Human Capital Management for Admin, HR and Employees, with a built-in Superadmin

Version 1.0 | Hackathon date: 24 Oct 2026 | Team: 2 backend, 2 frontend

1. Purpose and Scope

HCM Lite is a web-based HR system that an organization uses for daily people operations: employee records, attendance, leave, payroll and analytics, with an AI assistant that answers from real data.

This document extends the hackathon brief with these additions requested by the team:

- A default **Superadmin** account created automatically on first start.
- Support for **multiple Superadmin users**, all with entire system access.
- A changeable password policy.
- A full **validation** strategy.
- A **system testing** plan.
- **SEO from a human perspective**: pages that people can find, read, understand and use.

The system starts with **no dummy data**. Only the Superadmin account exists on a fresh install. Every other record comes from people using the app.

2. Roles and Access Model

Role	Who	Access summary
SUPERADMIN	Platform owners, IT lead	Entire system: every module, every record, all settings, user management of all roles including other Superadmins, audit log, payroll finalize
ADMIN	Company administrator	Company level as in the brief: setup, HR logins, finalize payroll, audit log
HR	People operations	Register employees, attendance board, leave approvals, payroll draft, HR dashboard

Role	Who	Access summary
EMPLOYEE	Staff	Own data only: check-in, leave, payslips, profile, assistant

Rules:

1. Superadmin is a superset of Admin. Anything Admin can do, Superadmin can do.
2. Superadmin can additionally create, deactivate and reset **any** user, including Admin and other Superadmins.
3. Admin and HR **cannot** create Superadmin users.
4. Access is enforced **on the server for every request**. Hiding a button is never enough.
5. Employees only read rows whose `employee_id` matches their own.
6. The AI assistant always runs with the same role and data scope as the person asking.

2.1 Default Superadmin

On first boot, if the users table has no Superadmin, the system creates one:

Field	Value
Username	`superadmin`
Initial password	`superadmin@123`
Role	SUPERADMIN
must_change_password	true
is_system_default	true

Important behavior:

- The password is **hashed with bcrypt** (cost 12). It is never stored or logged in plain text.
- The seed runs **once and only if no Superadmin exists**. Restarting the server never resets a changed password.
- Initial username and password can be overridden by environment variables `SUPERADMIN_USERNAME` and `SUPERADMIN_PASSWORD`. If they are not set, the defaults above are used.
- Because this default is publicly known (it is written in this document), the first login **forces a password change** before any other screen is available.

- The README must warn: change the default password immediately on any hosted deployment.
- The default account cannot be deleted. It can be deactivated only if at least one other active Superadmin exists.

2.2 Multiple Superadmins

- A Superadmin can create more Superadmin users from **Users > New user > Role: Superadmin**.
- Each new Superadmin gets a temporary password and must change it on first sign-in.
- All Superadmins have identical, full access.
- **Last-Superadmin guard:** the system refuses to deactivate, demote or delete the last remaining active Superadmin. The API returns 409 with a clear message.
- Every Superadmin action is written to the audit log with the acting user.

2.3 Changeable Password

Any user can change their own password via `POST /auth/change-password` (current password required). A Superadmin can reset another user's password, which sets a temporary password and `must_change_password = true`.

Password policy:

- Minimum 8 characters.
- At least one letter, one number and one special character.
- Must differ from the current password and from the username.
- Maximum 72 characters (bcrypt limit).
- Failed login lockout: 5 failed attempts locks the account for 15 minutes.
- On change, all existing refresh tokens for that user are revoked.

Note: the default `superadmin@123` itself passes the format rule (so it can be seeded), but the **new** password must not equal it.

3. Architecture

```
Frontend (React, responsive 375px+)
   |  REST + JSON + JWT
Backend API (Node/Express or similar)
   routes -> services (business rules) -> data layer
   |                          |
Database (PostgreSQL/MongoDB)    LLM API (tool calling, key on server only)
```

Layers:

- **Routes:** parse request, run validation, call a service. No business logic.
- **Services:** all rules live here (attendance status, leave checks, payroll math, metrics). The AI tools call the **same services**, so assistant answers always match dashboards.
- **Data layer:** queries and aggregation.
- **Middleware:** authentication (JWT), role guard, data-scope guard, rate limiting, audit hooks, error handler.

Suggested stack (any is allowed): React + Vite, Node + Express, PostgreSQL, JWT, bcrypt, Zod or Joi, Jest + Supertest, Playwright, PDFKit for payslips.

4. Database Schema

Tables below use SQL-style naming. For MongoDB, treat each as a collection.

4.1 Identity and Setup

users

Column	Type	Notes
id	uuid PK	
username	varchar(50) unique	lower-case, letters, digits, dot, underscore
email	varchar(255) unique, nullable	required for non-system users
password_hash	varchar	bcrypt

Column	Type	Notes
role	enum	SUPERADMIN, ADMIN, HR, EMPLOYEE
employee_id	uuid FK nullable	set for EMPLOYEE (and HR if also an employee)
is_active	boolean	default true
is_system_default	boolean	true only for seeded superadmin
must_change_password	boolean	default true
failed_attempts	int	default 0
locked_until	timestamp nullable	
last_login_at	timestamp nullable	
created_by	uuid FK nullable	
created_at, updated_at	timestamp	

refresh_tokens: id, user_id, token_hash, expires_at, revoked_at, created_at.

company: id, name, logo_url, created_at. Single row.

settings (single row, versioned): office_start (09:00), grace_minutes (15), half_day_hours (5), weekend_days (Sat,Sun), office_lat, office_lng, office_radius_m (150), working_hours_per_week (40), updated_by, updated_at.

settings_history: id, snapshot JSON, effective_from, changed_by. Used so rule changes apply to **new records only**.

departments: id, name unique, is_active.

designations: id, title, department_id nullable, is_active.

leave_types: id, name (annual, sick, casual, unpaid), yearly_quota (14, 8, 10, null for unlimited), is_paid.

holidays: id, date unique, name.

4.2 People

employees

Column	Type	Notes
id	uuid PK	
code	varchar unique	e.g. EMP-0001
first_name, last_name	varchar	
email	varchar unique	
phone, address	varchar	employee may edit these two
gender	enum	
date_of_birth	date	
department_id, designation_id	uuid FK	
join_date	date	ISO
exit_date	date nullable	
status	enum	probation, active, exited
manager_id	uuid nullable	

salary_structures: id, employee_id, basic (> 0), effective_from, set_by. Allowances are derived: HRA = 40% of basic, medical = 10% of basic, so gross = 1.5 x basic.

4.3 Attendance and Leave

attendance

Column	Notes
id, employee_id, date	unique (employee_id, date): second check-in returns 409
check_in_at, check_out_at	server-stamped
status	present, late, half_day, absent, missing_checkout, on_leave
worked_minutes	computed at check-out
source	web, camera, correction

Column	Notes
rule_snapshot	JSON of the settings used (grace, half-day hours)
lat, lng, accuracy_m, distance_m, face_count	camera check-ins only

attendance_corrections: id, employee_id, date, requested_check_in, requested_check_out, reason, status (pending, approved, rejected), decided_by, decision_comment, decided_at.

leave_requests: id, employee_id, leave_type_id, start_date, end_date, working_days, reason, status (pending, approved, rejected, cancelled), decided_by, decision_comment, decided_at, created_at.

4.4 Payroll

payroll_runs: id, month (YYYY-MM), department_id nullable, status (draft, finalized), total_gross, total_net, created_by, finalized_by, finalized_at. Unique on month. Once finalized, no updates.

payslips: id, run_id, employee_id, working_days, absent_days, unpaid_days, half_days, late_penalty_days, gross, deductions, tax, net, breakdown JSON, pdf_generated_at.

4.5 Communication, Audit, AI

announcements: id, title, body, posted_by, created_at, expires_at.

notifications: id, user_id, type, message, link, read_at, created_at.

audit_logs (append-only): id, actor_id, actor_role, action, entity, entity_id, before JSON, after JSON, ip, created_at. No update or delete endpoints exist. Database user for the app should have INSERT and SELECT only on this table.

ai_logs: id, user_id, feature (chat, summary, smart_leave, policy), prompt, tool_called, status, latency_ms, created_at.

4.6 Indexes

- attendance (employee_id, date) unique, attendance (date)
- leave_requests (employee_id, status), (start_date, end_date)
- employees (department_id, status), employees (email), (code)

- audit_logs (created_at), (actor_id), (entity, entity_id)
- users (username), (email)

5. Business Rules and Calculation Logic

5.1 Attendance Status

- Office start 09:00, grace 15 minutes. Check-in after 09:15 gives **late**.
- Worked time under 5 hours gives **half_day**.
- No record and no approved leave on a working day gives **absent** (computed on the fly for reporting, no nightly job required).
- Check-in without check-out at end of day gives **missing_checkout**.
- Approved leave on that date gives **on_leave**.
- Weekends (Sat, Sun) and public holidays are not working days.
- The server stamps all times. The client never sends a time.
- Status uses the settings snapshot saved on the record, so later setting changes do not rewrite history.

5.2 Camera Attendance (optional bonus)

The server rejects the check-in if any of these is true:

- Face count is not exactly 1.
- GPS accuracy is worse than 100 m.
- Haversine distance to office is greater than the Admin-set radius.

The server calculates distance and decides. The client only sends data. Store the result (distance, face count, source = camera), not the photo.

5.3 Leave

- Working days only: skip weekends and holidays.
- Annual quota 14, sick 8, casual 10, unpaid unlimited (skip balance check).
- Overlap with another pending or approved request returns **422**.
- Over-balance returns **422**.
- End date before start date returns **422**.
- Balance = quota - approved days in the calendar year.

- Employee can cancel while pending, or approved if start date is in the future.

5.4 Payroll

Inputs for a month: salary structure, attendance records, approved leave, holidays.

```
basic          = salary_structure.basic
HRA            = 0.40 x basic
medical        = 0.10 x basic
gross          = basic + HRA + medical = 1.5 x basic
working_days   = days in month - weekends - holidays
per_day        = gross / working_days

deduction_days = absent_days + unpaid_leave_days + 0.5 x half_days
                 (+ 0.5 x floor(late_count / 3) if late-penalty bonus is
enabled)
deductions     = per_day x deduction_days

proration      : for joiners or leavers mid-month, use only the days the
                 employee was employed; per_day divisor stays the month's
                 working days and payable days are reduced accordingly.

annual_gross   = gross x 12
annual_tax     = slab(annual_gross)
monthly_tax    = annual_tax / 12

net            = gross - deductions - monthly_tax
```

Tax slab on annual gross (illustrative, from the brief):

Annual gross	Annual tax
0 to 600,000	0
600,001 to 1,200,000	5% of amount above 600,000
1,200,001 to 2,400,000	30,000 + 15% of amount above 1,200,000
2,400,001 to 3,600,000	210,000 + 25% of amount above 2,400,000
Above 3,600,000	510,000 + 35% of amount above 3,600,000

Worked example: basic 100,000. Gross 150,000. Working days 22. Per day 6,818.18. One absent day and 2 half days gives 2 deduction days, so deductions 13,636.36. Annual gross 1,800,000, tax 30,000 + 15% x 600,000 = 120,000 a year, so 10,000 a month. Net = 150,000 - 13,636.36 - 10,000 = **126,363.64**.

Document these assumptions in the README: monthly tax is annual tax divided by 12, and the divisor is working days in the month.

Workflow: HR creates the draft, reviews, then **Admin or Superadmin finalizes**. A finalized run is locked: any update returns 409 and a change attempt is audited.

5.5 Metrics

Metric	Formula
Headcount	Active + probation on the end date
Attrition	Exits in period / average headcount x 100
Attendance rate	(present + late + missing_checkout + 0.5 x half_day) / (working-day records - on_leave)
Late %	late / (present + late)
Absenteeism	absent days / expected working days, per department
Leave utilization	Approved days / quota, per leave type and department
Payroll cost	Sum of gross and net by month and department
Average salary	Mean gross by department and designation

All metrics are computed on the server with database aggregation and filterable by date range and department.

6. API Route List

All routes are prefixed `/api/v1`. Every route except login, first-run status and health requires a valid JWT. Roles shown: S = Superadmin, A = Admin, H = HR, E = Employee. Superadmin is allowed on every route.

6.1 Auth

Method	Route	Roles	Purpose
POST	/auth/login	public	Username or email plus password. Returns tokens and `must_change_password`
POST	/auth/refresh	public	New access token
POST	/auth/logout	all	Revoke refresh token
POST	/auth/change-password	all	Current plus new password
GET	/auth/me	all	Profile, role, menu permissions
GET	/setup/status	public	Whether company setup is complete
POST	/setup/company	S, A	Company name and first settings (first-run wizard)

While `must_change_password` is true, every route except change-password, logout and me returns 403 with code `PASSWORD_CHANGE_REQUIRED`.

6.2 User Management

Method	Route	Roles	Purpose
GET	/users	S, A	List with search and paging
POST	/users	S (any role), A (HR, EMPLOYEE only)	Create user, temp password
PATCH	/users/:id	S, A	Edit email, role (within rights)
POST	/users/:id/reset-password	S, A	Set temp password, force change
POST	/users/:id/deactivate	S, A	Deactivate (last-Superadmin guard)
POST	/users/:id/activate	S, A	Re-activate

6.3 Organization Setup

Method	Route	Roles
GET/PUT	/settings	GET: S, A, H. PUT: S, A
CRUD	/departments	GET: S, A, H. Write: S, A
CRUD	/designations	GET: S, A, H. Write: S, A
CRUD	/leave-types	GET: S, A, H, E. Write: S, A
CRUD	/holidays	GET: all. Write: S, A

6.4 Employees

Method	Route	Roles	Purpose
GET	/employees	S, A, H	Search, filter, page (server-side)
POST	/employees	S, H	Register, creates login
GET	/employees/:id	S, A, H, E (own)	Employee 360
PATCH	/employees/:id	S, H	Edit details
PATCH	/me/profile	E	Own phone and address only
PUT	/employees/:id/salary	S, H	Salary structure
POST	/employees/import?dryRun=true	S, H	CSV validation, returns row, field, reason
POST	/employees/import	S, H	Commit import
GET	/employees/export	S, H	CSV export

6.5 Attendance

Method	Route	Roles	Purpose
POST	/attendance/check-in	E, H	Server-stamped. Second call returns 409

Method	Route	Roles	Purpose
POST	/attendance/check-out	E, H	
POST	/attendance/camera-check-in	E	Optional bonus
GET	/attendance/me?month=	E	Own calendar
GET	/attendance/board?date=	S, A, H	Daily board
POST	/attendance/corrections	E	Request with reason
GET	/attendance/corrections	S, A (view), H, E (own)	List
POST	/attendance/corrections/:id/decide	S, H	Approve or reject

6.6 Leave

Method	Route	Roles	Purpose
POST	/leave	E, H	Apply (422 on overlap or over-balance)
GET	/leave	S, A, H (all), E (own)	History
GET	/leave/balance	E, H	Own balance
POST	/leave/:id/cancel	E, H	Cancel own
POST	/leave/:id/decide	S, H	Approve or reject with comment

6.7 Payroll

Method	Route	Roles	Purpose
POST	/payroll/runs	S, H	Create draft for a month
GET	/payroll/runs/:id	S, A, H	Review
POST	/payroll/runs/:id/recalculate	S, H	Only while draft
POST	/payroll/runs/:id/finalize	S, A	Lock the month
GET	/payslips/me	E	Own payslips

Method	Route	Roles	Purpose
GET	/payslips/:id/pdf	S, A, H, E (own)	PDF download

6.8 Dashboards

Method	Route	Roles
GET	/dashboard/admin	S, A
GET	/dashboard/hr	S, H
GET	/dashboard/me	E
GET	/analytics/{headcount, attendance, leave, payroll}	S, A, H (filters: from, to, departmentId)

6.9 AI, Comms and Audit

Method	Route	Roles
POST	/ai/chat	all
POST	/ai/summarize	S, A, H
POST	/ai/smart-leave	E, H
GET/POST	/announcements	GET: all. POST: S, A, H
GET	/notifications	own
POST	/notifications/:id/read	own
GET	/audit-logs	S, A only
GET	/health	public

6.10 Standard Responses and Errors

Code	Meaning
200 / 201	Success / created

Code	Meaning
400	Malformed request
401	Not logged in or token invalid
403	Role or data scope denied, or password change required
404	Not found (also used to hide other employees' records)
409	Conflict: duplicate check-in, locked payroll, last Superadmin
422	Validation or business rule failure
429	Rate limit

Error body: `{ "error": { "code": "LEAVE_OVERLAP", "message": "You already have leave on these dates.", "fields": [{ "field": "start", "reason": "..." }] } }`. Messages are written for people, not developers.

7. AI Assistant Design

7.1 Principles

- The model **explains**. The database is the only source of truth.
- The API key is in `AI_API_KEY` on the backend only. The browser never calls the provider.
- The model never queries the database. It calls named tools that run the same services and permission checks as the REST API.
- The AI never saves data. Smart leave only pre-fills a form; the user confirms.
- Timeout 15 s, one retry on invalid JSON, then a friendly fallback to the normal form.
- About 20 requests per user per hour. Every call is written to `ai_logs`.
- Every answer carries an "AI-generated" label.

7.2 Tool Definitions

Tool	Parameters	Allowed roles	Scope
get_headcount	departmentId?, asOf?	S, A, H	Company
get_attendance_summary	from, to, departmentId?	S, A, H (company), E (own)	By role
get_late_comers	from, to, limit?	S, A, H	Company
get_leave_summary	from, to, departmentId?, leaveType?	S, A, H	Company
get_my_leave_balance	year?	all	Own
get_payroll_summary	month, departmentId?	S, A, H	Company (employees refused)
search_employees	query, departmentId?	S, A, H	Company, no salary for HR peers unless allowed

Example tool schema:

```
{
  "name": "get_late_comers",
  "description": "Employees ranked by number of late arrivals in a date
range.",
  "input_schema": {
    "type": "object",
    "properties": {
      "from": { "type": "string", "format": "date" },
      "to": { "type": "string", "format": "date" },
      "limit": { "type": "integer", "default": 5 }
    },
    "required": ["from", "to"]
  }
}
```

The backend injects the user's role and employee_id into every tool call. The model cannot override them. If a tool is not allowed for the role, the tool returns a permission error and the assistant politely refuses, for example: "I can't share other employees' salaries."

7.3 Other AI Features

- **Dashboard summary:** send only the KPI JSON for the current filters; ask for 3 to 5 bullets using only those numbers.
- **Smart leave:** "Need next Monday and Tuesday off for my sister's wedding" returns `{type, start, end, reason}`. The server validates the JSON schema and dates, then the form is pre-filled.
- **Policy Q&A:** Section 5 rules are in the system prompt. If the answer is not in the rules, the model says it does not know.
- **Roman Urdu (bonus):** add a line in the system prompt to understand and answer in the user's language.

8. Validation Strategy

Validate at three layers: the browser (fast feedback), the API (the real gate, using a schema library) and the database (constraints as the last safety net).

8.1 Field Rules

Field	Rule	Message shown to the person
Username	3 to 50 chars, a-z 0-9 . _	"Use 3 to 50 letters, numbers, dots or underscores."
Email	Valid format, unique, max 255	"Enter a valid email address." / "This email is already registered."
Password	See 2.3	"Use at least 8 characters with a letter, a number and a symbol."
Name	1 to 100 chars, letters, spaces, hyphen, apostrophe	"Please enter a name."
Phone	7 to 15 digits, optional +	"Enter a valid phone number."
Dates	ISO yyyy-mm-dd, real calendar date	"Enter a valid date."

Field	Rule	Message shown to the person
Leave dates	End on or after start, not in a closed year	"End date cannot be before start date."
Salary	Number greater than 0, max 2 decimals	"Salary must be greater than zero."
Office radius	10 to 5000 m	"Radius must be between 10 and 5000 meters."
Latitude / longitude	-90 to 90 / -180 to 180	"Enter a valid location."
Employee code	Unique, pattern EMP-0000	"This code already exists."
Month	YYYY-MM, not in the future for finalize	"Choose a valid month."

8.2 Cross-Field and Business Validation

- Department and designation must exist and be active.
- Join date not after exit date.
- Duplicate check-in returns 409.
- Leave overlap and over-balance return 422.
- Payroll for a month cannot be created twice, and finalized runs cannot be edited.
- Role assignment: Admin and HR cannot assign SUPERADMIN.
- Last active Superadmin cannot be deactivated or demoted.
- CSV import: required fields, unique code and email, valid email, known department, ISO date, salary > 0. The dry run returns **row, field, reason** and writes nothing.

8.3 Security-Oriented Validation

- Trim and normalize input; reject unknown fields.
- Parameterized queries only (no string-built SQL).
- Escape output to prevent XSS; set CSP and security headers (helmet).
- Rate limit login and AI endpoints.
- CORS limited to the frontend origin.
- Never return password hashes, tokens or stack traces.

8.4 Human-Friendly Validation UX

- Show errors next to the field, in plain words, and keep what the person typed.
- Validate on blur and again on submit; do not shout errors before the person has typed.
- Mark required fields; show password rules live while typing.
- Disable the submit button only while saving, with a visible loading state.
- Focus moves to the first invalid field. Errors are readable by screen readers (`aria-describedby` , `role="alert"`).

9. System Testing Plan

9.1 Test Layers

Layer	Tooling	What it covers
Unit	Jest	Pure rules: attendance status, working-day counter, payroll math, tax slab, Haversine, password policy
Integration / API	Supertest + test database	Each route: success, validation failure, role denial, scope denial
End-to-end	Playwright or Cypress	The full 5-minute demo script on a fresh database
Security	Manual plus scripted	RBAC matrix, token tampering, injection attempts
AI	Scripted prompts with a mocked provider, plus a few live checks	Tool selection, scoping, refusal, fallback
Accessibility and responsive	Lighthouse, axe, manual at 375px	Labels, contrast, keyboard use

9.2 Superadmin Test Cases

ID	Case	Expected
SA-01	Fresh install, start server	One user `superadmin` exists, role SUPERADMIN, password stored as bcrypt hash
SA-02	Restart server twice	No duplicate Superadmin created
SA-03	Login with `superadmin` / `superadmin@123`	Success, `must_change_password` true
SA-04	Call any business route before changing password	403 PASSWORD_CHANGE_REQUIRED
SA-05	Change password to weak value	422 with field errors
SA-06	Change password to valid new value	200, old tokens revoked, old password no longer works
SA-07	Restart server after password change	New password still works; default does not
SA-08	Superadmin creates second Superadmin	201, second user must change password on first login
SA-09	Second Superadmin opens every module	All routes allowed
SA-10	Admin tries to create Superadmin	403
SA-11	Deactivate the only active Superadmin	409
SA-12	Deactivate one of two Superadmins	200, deactivated user cannot log in
SA-13	Five wrong passwords	Account locked 15 minutes, clear message
SA-14	Superadmin resets an employee password	Temp password works once, change forced

ID	Case	Expected
SA-15	All Superadmin actions	Appear in audit log with actor, action, entity, before/after

9.3 Core Flow Test Cases

ID	Case	Expected
AT-01	Check-in at 09:10	status present
AT-02	Check-in at 09:20	status late
AT-03	Check-in twice same day	409
AT-04	Check-out after 4h	status half_day
AT-05	Settings changed, old record	Status unchanged (rule snapshot)
LV-01	Leave Fri to Mon	Counts 2 working days
LV-02	Leave spanning a holiday	Holiday not counted
LV-03	Overlapping request	422
LV-04	Sick leave 9 days (quota 8)	422
LV-05	Unpaid leave any length	Accepted
LV-06	Employee approves own leave	403
PY-01	Basic 100,000, no deductions	Gross 150,000
PY-02	Worked example in 5.4	Net 126,363.64
PY-03	Mid-month joiner	Prorated gross
PY-04	Finalize as HR	403
PY-05	Edit after finalize	409, audited
RB-01	Employee reads another employee id	404 or 403
RB-02	Employee asks AI for colleague salary	Polite refusal

ID	Case	Expected
RB-03	HR opens audit log	403
RB-04	Tampered or expired JWT	401
DS-01	Dashboard numbers vs records created in test	Exact match
AI-01	Ask "who came late most often this month"	Same result as dashboard top late-comers
AI-02	AI provider down	Friendly fallback, app keeps working
AI-03	Invalid JSON from model	One retry, then normal form

9.4 Quality Gates

- All unit and API tests pass in CI before deploy.
- At least 80% coverage on service-layer files (attendance, leave, payroll, auth).
- E2E demo script passes on a freshly migrated empty database.
- No high severity findings in the RBAC matrix check.
- Lighthouse accessibility score of 90 or higher on login and dashboard.

10. SEO from a Human Perspective

Most of HCM Lite sits behind a login, so search engines should not index it. SEO here has two goals: let the **public pages** be easy to find and understand, and make the **whole app** easy for real people to use, read and share. These practices help both search engines and humans.

10.1 Public vs Private Pages

Page	Indexed?	Notes
Landing / about page	Yes	Explains what HCM Lite does, in plain words

Page	Indexed?	Notes
Login page	Yes (title only)	Clear title, no sensitive details
Help / FAQ (optional)	Yes	Answers real questions people ask
Dashboard, employees, payroll, everything after login	**No**	`noindex, nofollow`, plus auth. Never expose personal data to crawlers

Use `robots.txt` to block private paths, and also send the `X-Robots-Tag: noindex` header and a `<meta name="robots" content="noindex">` tag on private routes. Provide `sitemap.xml` listing only public pages.

10.2 On-Page Basics for Humans

- **Title per page**, written for a person: "Leave Requests | HCM Lite", under 60 characters, unique on every screen (also helps browser tabs and history).
- **Meta description**, 120 to 155 characters, in a natural sentence.
- **One H1 per page**, with H2 and H3 in a logical order. Do not skip levels.
- **Plain language:** use "Time off" or "Leave" the way staff say it; avoid internal jargon on screens and in error messages.
- **Readable URLs:** `/leave/requests`, `/payroll/2026-10` rather than `/page?id=83&x=2`.
- **Open Graph and Twitter tags** on public pages, so a shared link shows a clear title and image.
- **Favicon and app name** so the tab is recognizable.
- **Canonical URL** on public pages to avoid duplicates.
- **Structured data (JSON-LD)** `SoftwareApplication` or `Organization` on the landing page.

10.3 Performance and Experience

- Fast first load: code-split per module, compress assets, lazy-load charts, size images.
- Target Core Web Vitals on public pages: LCP under 2.5 s, CLS under 0.1, INP under 200 ms.
- Responsive from 375px with tap targets of at least 44px.
- Use HTTPS only; redirect HTTP to HTTPS.

- Single-page app: use a proper routing library with real URLs and update `<title>` on every route change. Pre-render or server-render the landing page so crawlers see content without running scripts.
- PWA (bonus): manifest and service worker for install and offline shell.

10.4 Accessibility (the human side of SEO)

- Every input has a visible label; icons have text alternatives.
- Color contrast meets WCAG AA (4.5:1 for body text).
- Full keyboard navigation with a visible focus outline.
- Skip-to-content link; landmarks (`header`, `nav`, `main`, `footer`).
- Images have meaningful `alt` text (empty alt for decorative ones).
- Loading, empty and error states with clear wording, for example: "No leave requests yet. Apply for leave to see it here."
- Language attribute set (`lang="en"`), with room for Urdu or Roman Urdu text.

10.5 Content Checklist

- Each public page answers one clear question a person would ask.
- Headings describe content; link text is meaningful ("Download payslip", not "Click here").
- No duplicate titles or descriptions.
- 404 page that helps: explains the problem and links back to the dashboard or home.

11. Security Checklist

- bcrypt cost 12; JWT access token 15 min, refresh token 7 days, stored hashed and revocable.
- Role and data-scope middleware on every route; automated RBAC matrix test.
- Default Superadmin must change password on first login; startup log warns if the default password is still in use.
- Audit log is append-only; all Superadmin and finalize actions recorded.
- Secrets only in environment variables. Commit `.env.example` only.
- Helmet headers, CORS allow-list, login and AI rate limits, request size limits.
- AI key only on the server; tools reuse service-level permission checks.

12. Environment and Deployment

`.env.example`:

```
PORT=4000
DATABASE_URL=postgres://user:pass@host:5432/hcm
JWT_SECRET=change-me
JWT_REFRESH_SECRET=change-me-too
SUPERADMIN_USERNAME=superadmin
SUPERADMIN_PASSWORD=superadmin@123
AI_API_KEY=
AI_MODEL=
FRONTEND_ORIGIN=http://localhost:5173
```

Deployment steps:

1. Deploy a hello-world backend and frontend early to confirm hosting works.
2. Set environment variables in the host's settings, not in the repo.
3. Run migrations, then start the app. The seed creates the Superadmin if none exists.
4. Open the live URL, log in as Superadmin, change the password, complete company setup.
5. Run the E2E demo script against the hosted URL.

13. Build Plan and Team Split

Phase	Work	Owner
1. Foundation	Schema, seed Superadmin, auth, role and scope middleware, audit hooks	Backend A
2. Org and people	Setup, users, departments, holidays, employees, CSV import	Backend A + Frontend A
3. Attendance and leave	Rules, check-in, leave checks, approvals	Backend B + Frontend A
4. Payroll	Salary, draft, finalize lock, payslip PDF	Backend B + Frontend B
5. Dashboards	Aggregations and charts	Backend B + Frontend B

Phase	Work	Owner
6. AI	Chat with 7 tools, summary, smart leave, policy Q&A	Backend B + Frontend B
7. Comms and polish	Announcements, notifications, empty and error states, SEO tags	Frontend A + B
8. Testing and docs	Run test plan, README, rehearse demo	All
9. Bonus	Browser camera check, Roman Urdu, dark mode	If time remains

Agree the API contract (Section 6) in the first 30 minutes so frontend can build against mocks.

14. README Template

```
# HCM Lite
AI-assisted HR system for Admin, HR and Employees.

## Live demo
- App: <hosted url>
- Repo: <repo url>

## Stack
React, Node/Express, PostgreSQL, JWT, bcrypt, <LLM provider>

## Run locally
1. cp .env.example .env and fill values
2. npm install
3. npm run migrate
4. npm run dev

## First login (default Superadmin)
Username: superadmin  Password: superadmin@123
You are forced to change the password on first login.
Change it immediately on any hosted deployment.

## Roles
Superadmin (full access, can create more Superadmins), Admin, HR, Employee.
```

```
## AI setup
Set AI_API_KEY and AI_MODEL in the environment. The key never reaches the
browser.

## Tests
npm test (unit + API), npm run e2e (demo script)

## Assumptions
- Monthly tax = annual tax / 12
- Payroll per-day divisor = working days in the month
- Absence is computed on the fly

## Done / not done
<checklist of the 25 functionalities and bonuses>
```

15. Five-Minute Demo Script

1. Fresh install: log in as `superadmin`, change the password.
2. Superadmin sets company, office hours and location, a department and leave types, creates a second Superadmin and an HR login.
3. HR registers two or three employees. One signs in, changes the temporary password and checks in.
4. Employee types a leave request in plain words, confirms the pre-filled form. HR approves it.
5. HR opens the dashboard, clicks Summarize, asks the assistant who is on leave, and runs payroll. Superadmin finalizes it.
6. Employee downloads the payslip and asks for a colleague's salary (politely refused).
7. Superadmin shows the audit log.

16. Open Decisions

- Whether Admin remains a separate role or Superadmin replaces it in the demo (this document keeps both).
- Final database choice (PostgreSQL recommended for aggregation-heavy dashboards).
- Which LLM provider and model for the free tier.
- Whether the late-penalty, face matching and PWA bonuses are attempted.